GITLAB-CICD

GITLAB-CICD

GitLab CI/CD is GitLab's built-in automation system.

What is .gitlab-ci.yaml

It is basically a file in the root directory of the project

When you push code, GitLab reads this file and understands:
"What jobs should I execute, in what order, and how?"

GITLAB RUNNER

GitLab itself does not execute your shell commands directly.

Gitlab says "I have a job that needs to run."

A GitLab Runner actually executes that job.

Think of it like these:

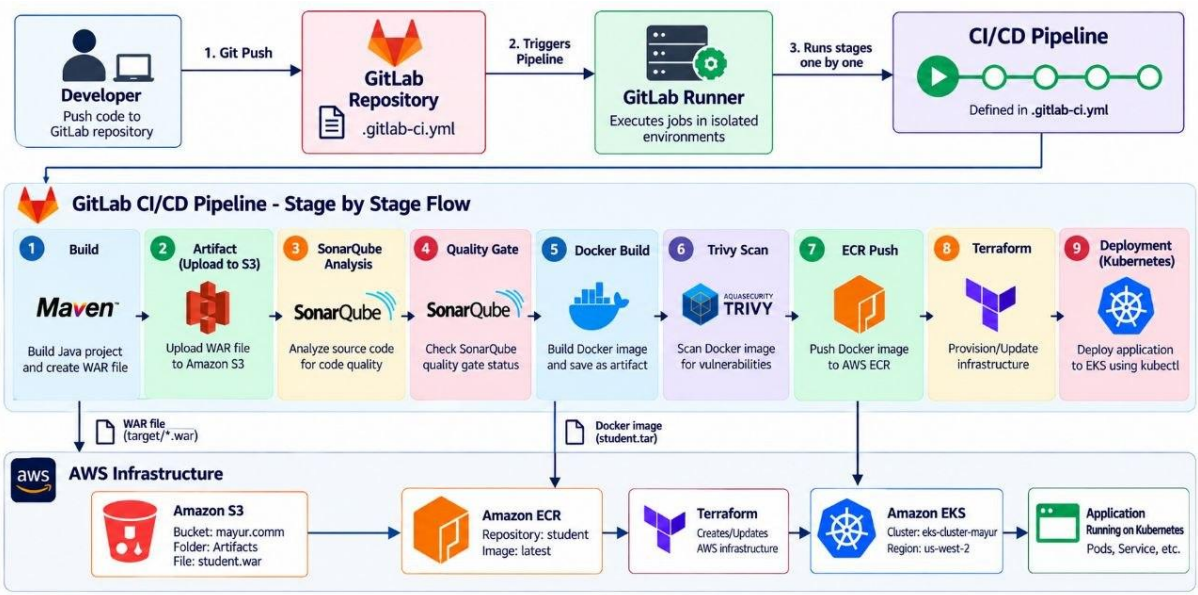
```
GitLab
  |
  | "Run this job"
  |
GitLab Runner
   |
   ├── download source code
   ├── prepare environment
   ├── execute script
   ├── collect artifacts
   └── send result to GitLab
```

TYPES OF GITLAB RUNNER

1. GitLab-hosted Runner
2. Self-managed Runner

1. Gitlab-hosted Runner
    1. GitLab provides the Runner infrastructure.
    2. You don't install the Runner yourself.
    3. GitLab provides the execution environment.

2. Self-managed Runner
    1. You install the GitLab Runner yourself.
       Example (EC-2 / On-premises server)
    2. Then GitLab sends jobs to your Runner.



REPOSITORY LINK

https://gitlab.com/mayurpandit25/student-management.git

GITLAB END-TO-END PIPELINE

1.CREATE A FILE

.gitlab-ci.yaml

```
stages:
  - build
  - artifact
  - sonar
  - quality-gate
```

```
  - docker-build
  - trivy-scan
  - ecr-push
  - terraform
  - deployment

# BUILD STAGE

build:
  stage: build

  image:
    name: maven:3.9-eclipse-temurin-21

  script:
    - mvn clean package

  artifacts:
    paths:
      - target/*.war
    expire_in: 1 hour

# UPLOAD WAR TO S3

upload-to-s3:
  stage: artifact

  image:
    name: alpine:latest

  dependencies:
    - build

  before_script:
    - apk add --no-cache aws-cli

  script:
    - aws s3 cp target/*.war s3://mayur.comm/Artifacts/student.war

# SONARQUBE ANALYSIS

sonarqube:
  stage: sonar
```

```
  image:
    name: maven:3.9-eclipse-temurin-21

  script:
    - mvn clean compile
    - mvn sonar:sonar
      -Dsonar.token="$SONAR_TOKEN"
      -Dsonar.host.url="$SONAR_HOST_URL"

# QUALITY GATES

quality-gate:
  stage: quality-gate
  image:
    name: alpine:latest
  before_script:
    - apk add --no-cache curl jq
  script:
    - |
      STATUS=$(curl -s -u "${SONAR_TOKEN}:" \

"${SONAR_HOST_URL}/api/qualitygates/project_status?projectKey=com.school:student-
management" \
        | jq -r '.projectStatus.status')

      echo "SonarQube Quality Gate: $STATUS"

      if [ "$STATUS" != "OK" ]; then
        echo "Quality Gate failed!"
        exit 1
      fi

      echo "Quality Gate passed!"

# DOCKER BUILD

docker-build:
  stage: docker-build

  image:
    name: docker:latest

  services:
    - name: docker:dind
```

```
  variables:
    DOCKER_HOST: tcp://docker:2375
    DOCKER_TLS_CERTDIR: ""

  script:
    - docker --version
    - docker build --no-cache -t student:latest .
    - docker images
    - docker save student:latest -o student.tar

  artifacts:
    paths:
      - student.tar
    expire_in: 1 hour

# TRIVY IMAGE SCAN

trivy-scan:
  stage: trivy-scan

  image:
    name: alpine:latest

  services:
    - name: docker:dind

  variables:
    DOCKER_HOST: tcp://docker:2375
    DOCKER_TLS_CERTDIR: ""

  dependencies:
    - docker-build

  before_script:
    - apk add --no-cache docker-cli curl
    - wget -qO-
https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sh -s
-- -b /usr/local/bin

  script:
    - docker --version
    - trivy --version
```

```
    - docker load -i student.tar
    - docker images
    - trivy image --scanners vuln --severity HIGH,CRITICAL --exit-code 0 student:latest

# PUSH IMAGE TO ECR

ecr-push:
  stage: ecr-push

  image:
    name: alpine:latest

  services:
    - name: docker:dind

  variables:
    DOCKER_HOST: tcp://docker:2375
    DOCKER_TLS_CERTDIR: ""

  dependencies:
    - docker-build

  before_script:
    - apk add --no-cache docker-cli aws-cli

  script:
    - docker --version
    - aws --version

    # Load image created in Docker Build stage
    - docker load -i student.tar

    #Push Image
    - aws ecr get-login-password --region us-west-2 |
      docker login --username AWS --password-stdin 822920903800.dkr.ecr.us-west-2.amazonaws.com
    - docker tag student:latest 822920903800.dkr.ecr.us-west-2.amazonaws.com/student:latest
    - docker push 822920903800.dkr.ecr.us-west-2.amazonaws.com/student:latest

# TERRAFORM

terraform:
  stage: terraform

  image:
    name: hashicorp/terraform:latest
    entrypoint: [""]

  script:
```

```
    - terraform --version
    - terraform init
    - terraform validate
    - terraform apply -auto-approve

# KUBERNETES DEPLOYMENT

deployment:
  stage: deployment

  image:
    name: alpine:latest

  before_script:
    - apk add --no-cache aws-cli curl kubectl

  script:
    - aws --version
    - kubectl version --client

    - aws eks update-kubeconfig --name eks-cluster-mayur --region us-west-2

    - kubectl apply -f k8s/

    - sleep 30

    - kubectl get pods
    - kubectl get svc -o wide
```

Where does AWS authentication come from

You don put cred inside the yaml

1. AWS_ACCESS_KEY_ID: xxx
2. AWS_SECRET_ACCESS_KEY: xxx

Instead, we configure these in
GitLab
→ Project (student-management)
→ Settings
→ CI/CD
→ Variables

Add these into the variables section

1. AWS_ACCESS_KEY_ID
2. AWS_SECRET_ACCESS_KEY
3. AWS_DEFAULT_REGION
4. SONAR_TOKEN
5. SONAR_HOST_URL
